



JAVA PROJECT SEM 4

Title of the Project:

FraudGuard

**A Banking Transaction Analyzer
with Real-Time Fraud Detection**

Group Members:

- Swaraj Deogirkar (24070126179)
- Tejas Kale (24070126189)
- Hardik Gulati (24070126063)
- Deep Thadeshwar (24070126050)

Mentor

Mr. Mayur Gaikwad

INDEX

- 1.Title
- 2.Introduction
- 3.Problem Statement
- 4.Proposed Solution
- 5.OOPs Design Overview
- 6.Concepts Used
- 7.Methodology
- 8.Module Breakdown
- 9.Timeline
- 10.Conclusion

INTRODUCTION

Background

- Digital banking & UPI have massively increased transaction volume per second
- Rise in fraud — unauthorized transfers, account takeovers, abnormal spending patterns

Domain / Application Area

- Banking & Financial Systems
- FinTech Security Solutions
- Fraud Detection & Risk Management

Motivation

- Manual/static fraud detection is reactive and unscalable
- Need for an automated system that continuously monitors transaction behavior in real time

PROBLEM DEFINITION

Problem Statement

- Design a Java-based banking system that automatically detects suspicious transactions using rule-based risk evaluation and behavioral profiling

Existing Issues / Gaps

- Just processes transactions.
- Does not continuously monitor behavior.
- Uses static rules only.
- Cannot explain why something is flagged.

Target Users

- Banking institutions
- Fraud monitoring teams
- Cybersecurity analysts
- Customers

PROJECT SCOPE

INCLUDED	EXCLUDED
ACCOUNT MANAGEMENT	REAL BANK API INTEGRATION
TRANSACTION PROCESSING	DATABASE PERSISTENCE
RULE - BASED FRAUD DETECTION	MOBILE UI

PROPOSED SOLUTION

FraudGuard is a rule-based fraud detection engine that processes transactions, evaluates risk, assigns fraud scores, and flags high-risk activity — all in real time via a background thread.

Feature	Function
Large Transaction Detection	Flags amounts exceeding defined threshold
Rapid Transaction Monitoring	Detects 3+ transactions in under 60 seconds
Odd Hours Detection	Flags transactions between 12AM–4AM
Velocity Check	Flags if daily withdrawal > 50% of balance
Risk Score Classification	Low (0–39) / Medium (40–69) / High (70–100)

Innovation / Uniqueness

- Risk scoring over binary block/allow — mirrors real fraud engines
- Transparent alerts — explains exactly which rule triggered the flag

OOP OVERVIEW

How OOP is Applied

- Every entity — Account, Transaction, FraudRule — is modeled as an object
- Each class has a single responsibility (Single Responsibility Principle)
- Encapsulation protects sensitive financial data from direct access
- Interfaces define contracts between the banking service and fraud engine
- Main service acts as a Facade, hiding internal thread and rule complexity

Justification for OOP

- Mirrors the real banking domain — Account, Transaction, Rule map to real entities
- Polymorphism allows new fraud rules to be added without changing existing logic
- Makes the system scalable, maintainable, and extensible

Encapsulation

DATA PROTECTION LAYER



Classes: BankAccount,
Transaction,
TransactionProfile

Implementation: Private
fields, controlled via
getters/setters, sensitive
data cleared after use



Why Important: Prevents
unauthorized modification
of financial data

Inheritance

TRANSACTION LAYER



Classes: Transaction (Parent), Deposit,
Withdrawal, Transfer (Children)

Implementation:
Custom exceptions **inherit**
behavior from the standard
exception class.

Why Important: Avoids
duplication, allows transaction-
type specific rules

Polymorphism

FLEXIBILITY LAYER



Classes:

**FraudRule (I), LargeAmountRule,
RapidFireRule, OddHoursRule,
VelocityRule**

Implementation:

**All rules implement
evaluate(Transaction t) — called
uniformly at runtime**

**Why Important: New rules added
without modifying the detection
engine**

Abstraction

SIMPLIFICATION LAYER



Classes:

**Account (Abstract), FraudRule
(Interface)**

Implementation:

**Complex fraud logic hidden inside
FraudDetectorService; only
evaluate() and process() exposed**

**Why Important: Reduces
complexity, improves clarity and
usability**

Development Approach

- Studied Producer-Consumer concurrency and BlockingQueue design
- Designed OOP architecture with clear module separation
- Implemented banking core first, then fraud rules, then threading independently
- Focused on correct concurrent design over unnecessary complexity

IDE / Software Environment

- Java Development Kit (JDK 17+)
- IntelliJ IDEA / Eclipse IDE
- Git & GitHub for version control
- Command-line interface for testing execution

Tools & Technologies Used:

Java (Core + OOP)

Multithreading

File Handling for logs

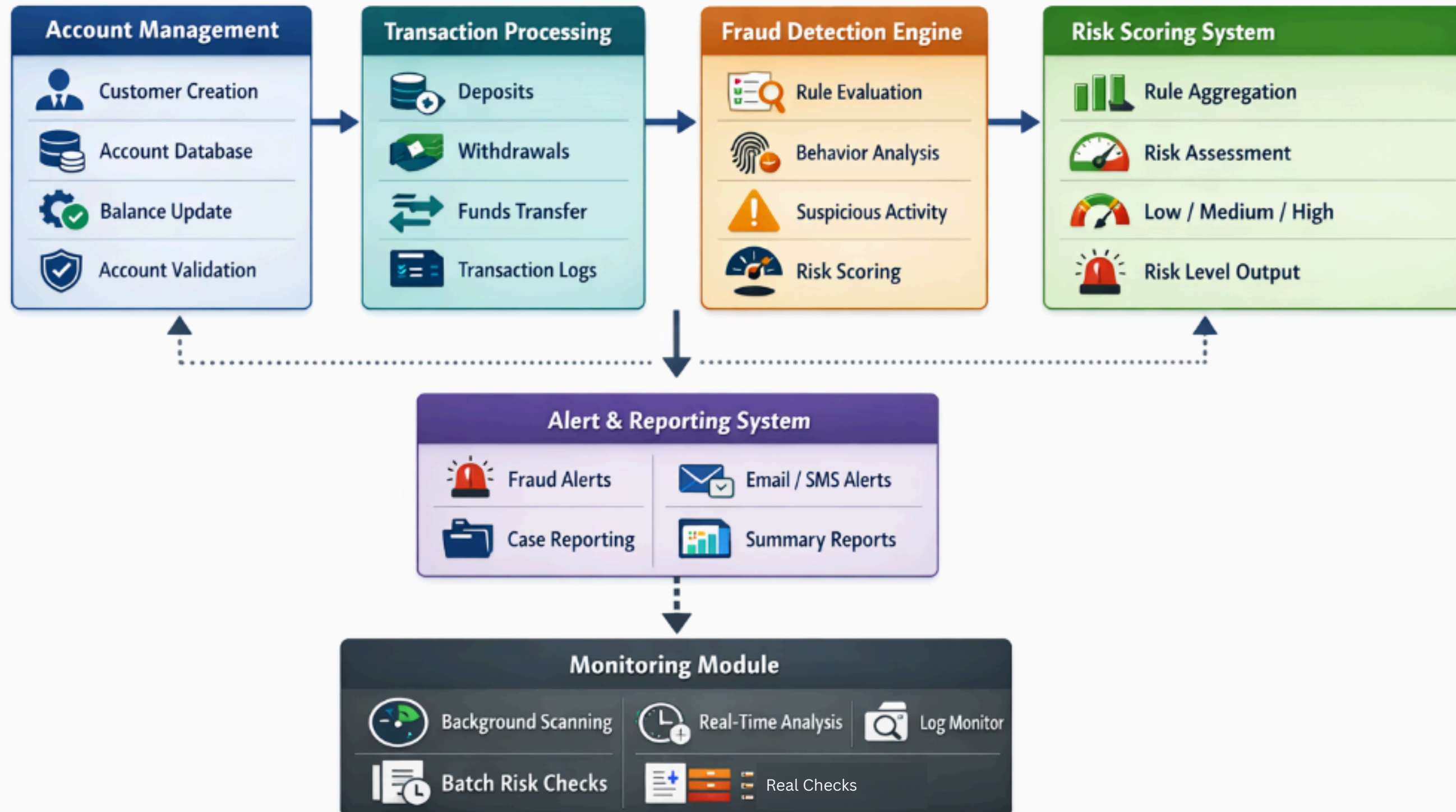
Data Structures (Lists, HashMaps)

MODULE BREAKDOWN

MODULE 1	Account Management	Handles customer creation and balance updates.
MODULE 2	Transaction Processing	Processes deposits, withdrawals, and transfers.
MODULE 3	Fraud Detection Engine	Evaluates fraud rules and calculates risk score.
MODULE 4	Risk Scoring System	Assigns risk levels based on cumulative rule scores.
MODULE 5	Alert & Reporting System	Generates fraud alerts and summary reports.
MODULE 6	Monitoring Module	Background transaction scanning

MODULE BREAKDOWN

Fraud-Guard System Architecture



TIMELINE

- Phase 1: Requirement analysis & design and Core Java & OOP implementation.
- Phase 2: Fraud detection logic & monitoring and Testing & documentation.

Milestones Achieved

- System architecture and class diagram finalized
- OOP structure implemented
- Core fraud rules coded and tested
- Producer-Consumer threading model operational

Upcoming Deliverables (Phase 2)

- CLI demo with live fraud flagging
- fraud_log.txt output with timestamped entries

CONCLUSION

- FraudGuard applies OOP and Java concurrency to a real financial domain problem
- Producer-Consumer via LinkedBlockingQueue — no missed transactions, no CPU waste
- Risk scoring + explainable alerts mirrors real banking fraud engine behavior
- Modular rule design — new rules or ML scoring can be added without redesign

Future Enhancements

- Machine learning-based anomaly detection
- Database integration (JDBC/SQLite)
- REST API layer for external client simulation
- Real-time dashboard for live transaction monitoring

Github Repository

<https://github.com/Hardik25-iso/FraudGuard>